CS 474 - Computer Security · Class 7

Secrets & Identities: Applied Cryptography, Authentication & Key Management

Thursday, November 5, 2026 · Reading: Easttom, Ch. 8

STUDENT EDITION

From the Industry to the Classroom — Building the IT's Future, Today and Together !

The hour — five lessons

Lesson 1 Crypto vocabulary and classical ciphers

Lesson 2 Symmetric encryption and hashing

Lesson 3 Asymmetric encryption, signatures and PKI

Lesson 4 Authentication and multi-factor authentication

Lesson 5 Key management in practice

The one FACT: your weekly work IS the project, and the project IS your exam preparation.

Scope & Legal Notice

- This is a computer-security course: you study cryptography to choose and configure strong, standard protection for systems you are authorized to defend.
- Every key, password and certificate here is for the fictional BlueFern Trading Inc. Never use real credentials, real private keys or anyone else's data.
- Never attempt to recover, bypass or weaken protection on a system you do not own or are not authorized in writing to test.
- Legal Notice, Terms of Use and Acceptable Use Policy: cyber-soho.github.io (sections 2, 3, 4.2/4.3, 5, 6.1/6.3/6.5, 8).
- Criminal Code of Canada s. 342.1 covers unauthorized use of a computer and the fraudulent interception of a password or key.

Lesson 1 — Crypto vocabulary and classical ciphers

- Plaintext, ciphertext, key, encryption, decryption
- Caesar and Vigenère ciphers + modular arithmetic
- Frequency analysis — why simple ciphers leak
- Kerckhoffs's principle: secrecy lives in the key

The five words everything is built on

- Plaintext → (encryption) → ciphertext → (decryption) → plaintext
- The key is the secret that controls both directions
- ORDER → RUGHU with key 3; the key is 3
- Cipher ≠ code; encoding (Base64) ≠ encryption (no secret, no confidentiality)

The Caesar cipher and modular arithmetic

- Number letters A=0 … Z=25
- Encrypt: (p + k) mod 26; decrypt: (c − k) mod 26
- “mod 26” wraps around like a clock after 12
- Only 25 useful keys → broken by trying all of them (brute force)

Frequency analysis breaks simple substitution

- A substitution cipher keeps each letter's frequency
- English: E, T, A, O, I, N most common; TH the top pair
- Count ciphertext letters → match to E, T, A … → read it
- Lesson: a cipher that keeps the plaintext's statistics leaks it

The Vigenère cipher — one step harder

- A repeating keyword gives several shifts
- ATTACK + KEY → KXRKGI (the two T’s differ)
- Defeats a single frequency count …
- … but Kasiski finds the key length → analyse each column

Kerckhoffs's principle

- A system must stay secure even if the algorithm is public
- Only the key is secret — and a key is easy to replace
- "Security through obscurity" fails when the method leaks
- Corollary: never invent your own cipher; use standard, public algorithms

Diagram — the Caesar wheel and why output must look random

Classical ciphers: the Caesar shift and why output statistics matter

Caesar cipher, key = 3 (plain row over cipher row)

A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z
D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C

O→R · R→U · D→G · E→H · R→U, so **ORDER → RUGHU**. Each letter number p becomes (p + 3) mod 26.

A cipher leaks when the ciphertext keeps the plaintext's letter-frequency shape

Plaintext letter counts



Caesar ciphertext — same shape, shifted → broken by frequency analysis



One-time pad



flat → no information leaks (provably secure, key used once)

Real-life example 1 — the developer’s note

Task. Encrypt a short BlueFern instruction three ways - a Caesar shift, a Vigenère keyword, and a one-time pad of truly random shifts - and show, by counting letters, which outputs still carry the fingerprint of the plaintext and which do not.

How it relates. This is Lesson 1 made measurable: the vocabulary of 1.1, the Caesar rule and modular arithmetic of 1.2, frequency analysis as the breaker of 1.3, the repeating keyword of 1.4, and the reason 1.5 says “do not invent your own” - a cipher is only as good as the flatness of its output statistics.

Deliverable. Feeds Exercise 7.1 (Caesar + Vigenère problems), Binder Part 7.

Lesson 2 — Symmetric encryption and hashing

- AES and why key length sets the brute-force cost
- Cryptographic hash functions and the avalanche effect
- Storing passwords: unique salt + slow hash
- HMAC — integrity and authenticity with a shared key

Symmetric encryption — one shared key

- Same key encrypts and decrypts
- AES: FIPS 197; 128-bit blocks; 128/192/256-bit keys
- Use an authenticated mode (GCM), never ECB
- Fast → right for bulk data (disk, database, TLS); wrong for passwords

Key length is the whole game for brute force

- An n-bit key has 2^n values; each bit doubles the work
- DES 56-bit: broken in ~56 hours (EFF, 1998)
- AES-128: 2^128 ≈ 3.4×10^38 → infeasible (> age of universe)
- Rule: add bits to beat brute force

Cryptographic hash functions

- Any input → fixed-size digest (SHA-256 = 256 bits); one-way
- Pre-image, second-pre-image, collision resistance
- Avalanche: one bit in → ~half the bits out change
- MD5 (2004) and SHA-1 (2017) are broken — use SHA-256+

Storing passwords — salt and slow (1/2)

- Store a hash, never the password
- Problem 1: identical passwords share a digest; rainbow tables
- Fix: a unique random salt per password (stored in the clear)
- Salt defeats precomputed tables

Storing passwords — salt and slow (2/2)

- Problem 2: fast hashes allow billions of guesses/second
- Fix: a slow function — Argon2id, bcrypt, scrypt, PBKDF2
- Rule: unique-salted + slow; never plaintext, never “encrypted”, never a bare fast hash
- People reuse passwords → add a second factor (Lesson 4)

MAC and HMAC — integrity + authenticity

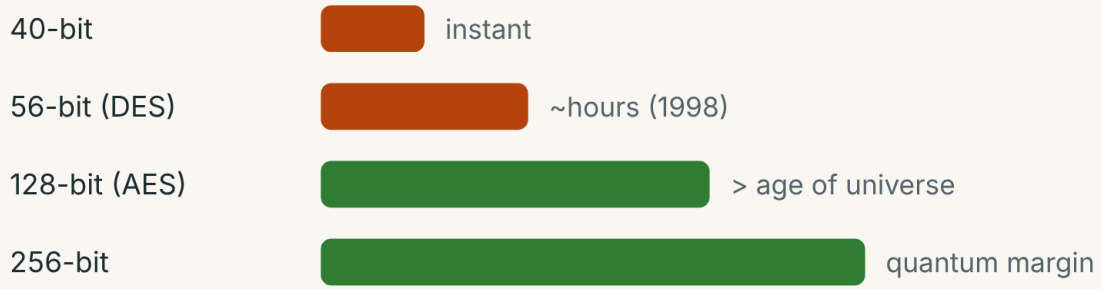
- Encryption hides content; it does not prove who sent it or that it is intact
- A MAC is a keyed tag over the message
- HMAC (RFC 2104): the standard MAC from a hash (HMAC-SHA256)
- Used inside TOTP codes (L4) and every TLS record (L5)

Diagram — storing a password; and brute-force vs key length

Storing a password: unique salt + slow hash



Brute force doubles with every key bit — why length wins



A salt defeats precomputed rainbow tables; slowness defeats large-scale guessing.

Real-life example 2 — the stolen password file

Task. Take a small table of BlueFern staff passwords and store it three ways - in plaintext, as a bare fast SHA-256, and as a unique-salted Argon2/bcrypt-style slow hash - then simulate an attacker who steals the file and measure how many accounts fall, and how fast, under each scheme.

How it relates. Hash properties and the avalanche effect (2.3), and the salt-and-slow rule (2.4), made measurable; it also builds directly on the password maths of Class 3 (keyspace and dictionary attacks).

Deliverable. Feeds Exercise 7.2 (password-storage specification), Binder Part 7.

Lesson 3 — Asymmetric encryption, signatures and PKI

- Public and private keys solve key sharing
- RSA and Diffie-Hellman (toy numbers)
- Digital signatures: authorship for everyone
- Certificates, CAs and the chain of trust behind the padlock

The key-sharing problem → public keys

- Symmetric needs a shared key already in place
- 38 people pairwise = 38×37/2 = 703 keys
- Key pair: public (shared) + private (secret)
- Encrypt to public / decrypt with private; sign with private / verify with public
- Slow → used to set up a symmetric key (hybrid, as in TLS)

RSA with toy numbers

- p=5, q=11 → n=55, φ=40
- public e=3; private d=27 (3×27=81≡1 mod 40)
- Encrypt m=9: 9^3 mod 55 = 14
- Decrypt: 14^27 mod 55 = 9
- Security: factoring a large n is infeasible

Diffie-Hellman with toy numbers

- Public p=29, g=2
- Alice a=5 → A=3; Bob b=9 → B=19
- Shared: B^a = A^b = 21 (mod 29)
- Eavesdropper cannot — discrete-log problem
- ECDHE (ephemeral) → forward secrecy

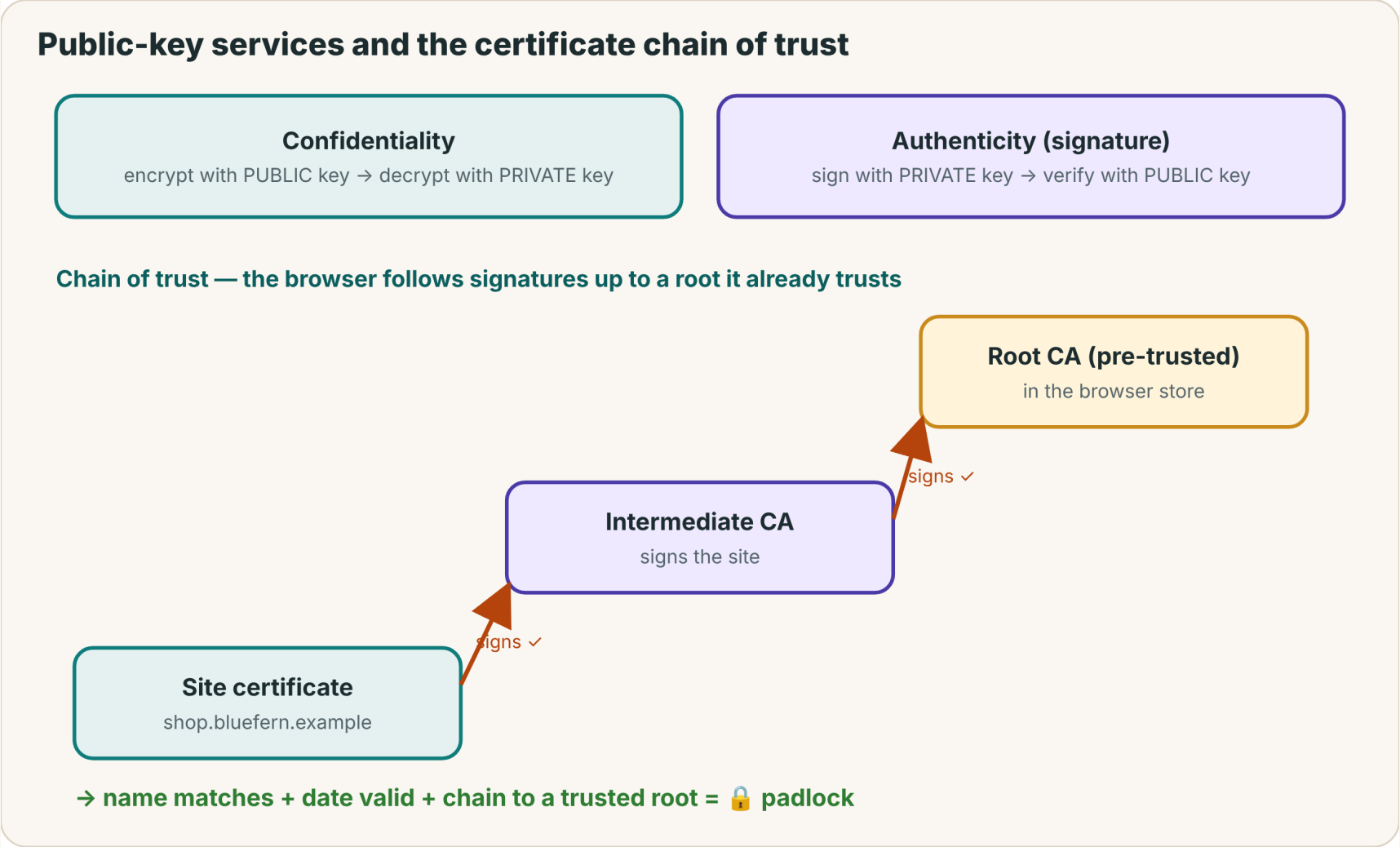
Digital signatures

- Hash the message, transform the digest with the PRIVATE key
- Verify with the PUBLIC key
- Gives integrity + authenticity + non-repudiation
- A MAC convinces key-holders; a signature convinces everyone

Certificates, CAs and the chain of trust

- A certificate binds a public key to a domain, signed by a CA
- Browser follows site → intermediate → trusted root
- Name matches + date valid + chain verifies → padlock
- DV certificates (Let’s Encrypt) are free and automatic; revoke via CRL/OCSP

Diagram — public-key services and the chain of trust



Real-life example 3 — the padlock on the web shop

Task. Walk a single customer's first HTTPS connection to shop.bluefern.example end to end: the browser receives the site's certificate, verifies the CA's signature up the chain to a trusted root, checks the name and expiry, agrees a fresh session key by ECDHE, and only then sends the order over AES - and see what breaks if the certificate is self-signed, expired, or for the wrong name.

How it relates. Public keys and the key-sharing problem (3.1), DH/ECDHE and hybrid encryption (3.2), the signature that makes a certificate trustworthy (3.3), and the certificate / CA / chain of trust (3.4) - all in one connection.

Deliverable. Feeds Exercise 7.2 (TLS certificate plan), Binder Part 7; reused in Part 8 (Class 10).

Lesson 4 — Authentication and MFA

- The three authentication factors
- Why MFA defeats a stolen password
- How a TOTP code and a passkey work
- SSO and Kerberos in one picture

Identification, authentication, three factors

- Identify = claim; authenticate = prove; authorize = allow
- Knowledge: password, PIN (guessed, phished, reused)
- Possession: phone, security key (lost, stolen)
- Inherence: fingerprint, face (cannot be reset if leaked)

Multi-factor authentication — two beats one

- MFA = two or more DIFFERENT factors
- Password + security question = one factor twice (not MFA)
- A stolen password alone no longer logs in
- Ranking: SMS (restricted) < app TOTP < passkey (phishing-resistant)

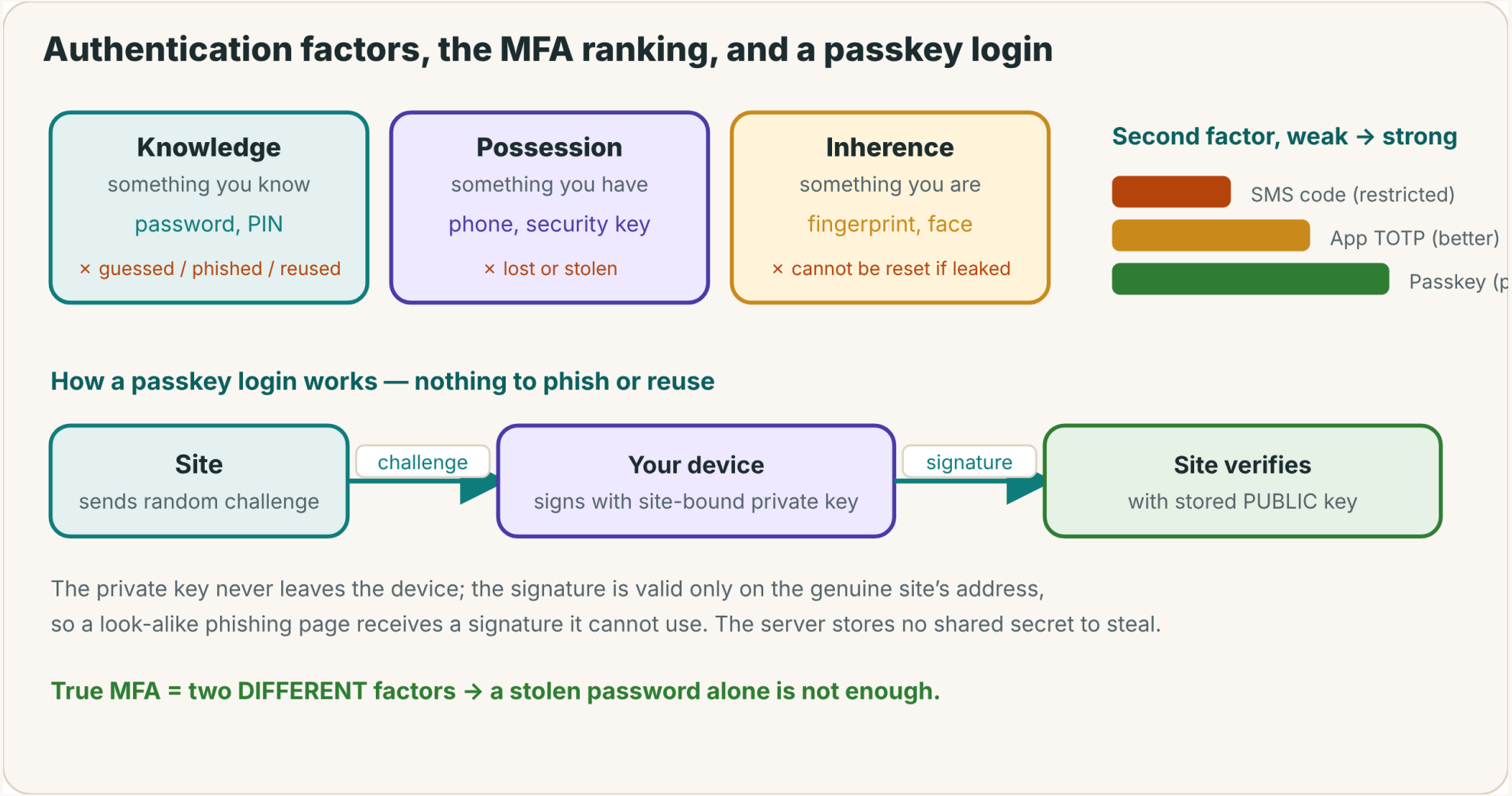
How TOTP and passkeys work

- TOTP: HMAC of the current 30-second step with a shared secret
- Same secret + same clock → same 6 digits; still phishable
- Passkey: a per-site key pair; private key stays on the device
- Signature bound to the real site → nothing to phish, steal or reuse

SSO and Kerberos in one picture

- SSO: authenticate once, reach many apps via a trusted token
- Benefit: one place for MFA; risk: a high-value central target
- Kerberos: a KDC issues a Ticket-Granting Ticket, then service tickets
- Password used once; tickets time-limited → clocks must be synced

Diagram — factors, the MFA ranking, a passkey login



Real-life example 4 — the stolen admin password

Task. Replay the Lesson 2 breach - the attacker has cracked BlueFern's one shared administrator password - against four login designs (password only, password + SMS code, password + TOTP app, passkey) and show, step by step, where the attacker is stopped and why, so BlueFern can decide which factor to put on which account first.

How it relates. The three factors (4.1), why two different factors beat one and the phishing-resistance ranking (4.2), and how a TOTP code and a passkey are produced and verified (4.3).

Deliverable. Feeds Exercise 7.2 (MFA roll-out plan), Binder Part 7.

Lesson 5 — Key management in practice

- The TLS handshake — the whole hour in one connection
- The key lifecycle: generate → … → revoke
- What a VPN protects, and what it doesn’t
- A note on the quantum future

The TLS 1.3 handshake — the hour in one connection

- 1 ClientHello → cipher list
- 2 Server sends its certificate (L3)
- 3 Browser verifies chain, name, date (L3)
- 4 ECDHE agrees a fresh key — forward secrecy (L3)
- 5 Switch to AES-GCM with a MAC (L2) → padlock

The key lifecycle — a key is not forever

- Generate from a CSPRNG (never from the clock)
- Distribute (the TLS key exchange); store with restricted access / HSM
- Rotate on a schedule and after exposure
- Revoke (CRL/OCSP) and destroy old copies

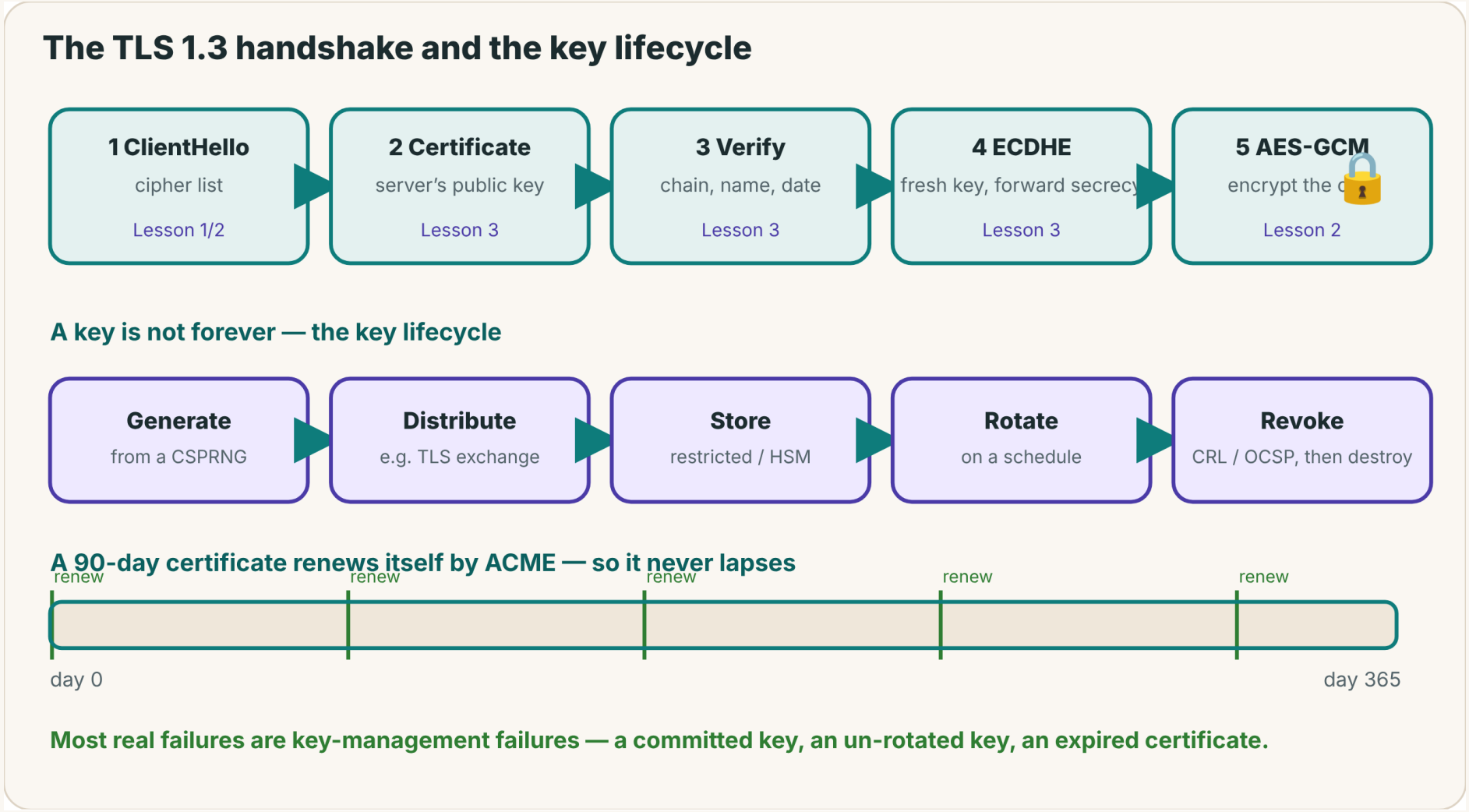
VPNs — everyday key management

- An encrypted tunnel over the public Internet (IPsec, WireGuard)
- Protects data in transit — not a compromised endpoint or a weak password behind it
- A VPN has its own keys → require MFA on it
- BlueFern: “a VPN exists; half skip it” → make it required

A note on the quantum future

- Symmetric/hashes: survive with larger keys (AES-256)
- RSA and DH/ECC: broken by a future quantum computer
- NIST PQC standards (ML-KEM, FIPS 203, 2024); deprecate after 2030, disallow after 2035 (IR 8547)
- “Harvest now, decrypt later” → long-lived secrets already at risk

Diagram — the TLS handshake and the key lifecycle



Real-life example 5 — one padlock, every idea

Task. Trace a complete secure session to BlueFern's web shop and then its key management over a year: the TLS 1.3 handshake step by step, the symmetric keys it derives, the 90-day certificate that renews automatically by ACME, and the rotation table that says what is changed, how often, and by whom - with a side panel showing what breaks when a step is skipped.

How it relates. The TLS handshake that unites the hour (5.1), the key lifecycle of generation/storage/rotation/revocation (5.2), the VPN as the same crypto applied to a connection (5.3), and a forward note on PQC (5.4).

Deliverable. Feeds Exercise 7.2 (TLS plan, VPN rule, key-rotation table), Binder Part 7; assembles into Milestone 1; reused in Part 8 (Class 10).

Hours 2–3 — Binder Part 7 (summary)

- Exercise 7.1 — crypto worksheet: Caesar, Vigenère, keys-needed, brute force, RSA toy, DH toy (show every step)
- Exercise 7.2 — identity & key-management design: MFA roll-out, password storage, TLS certificate plan, VPN rule, key-rotation table
- Due Wednesday, November 11, 23:59
- Then assemble Milestone 1 (Parts 1–7) — due Nov 18, the day before the Mid-Term

What you can now do

- Define the vocabulary and work the classical ciphers
- Reason about key length, hashing and correct password storage
- Work a toy RSA and Diffie-Hellman; explain signatures and certificates
- Choose and rank authentication factors; explain MFA, TOTP and passkeys
- Walk the TLS handshake and the key lifecycle